

Reproducible Detection of AI-Generated Faces via Open Handcrafted Features on PCA-Residual Images

[Author name]¹ and [Author name]¹

[Department / University name]
[email]

Abstract. The rapid proliferation of AI-generated face images poses concrete threats to identity verification, media integrity, and public trust. Removing dominant principal components from a face image and inspecting the residual exposes differences between real photographs and AI-generated faces. A recent study demonstrated this effect through a Kolmogorov-Smirnov test using proprietary fractal descriptors, but the closed-source toolkit prevents independent replication, and no classifier was ever trained on the representation. We replace the proprietary descriptors with three open alternatives computed on the PCA residual of each 256×256 grayscale face image: a 64-bin log-magnitude frequency histogram, a 59-bin rotation-invariant local binary pattern histogram, and a 64-bin Gaussian high-pass noise histogram. These are concatenated into a 187-dimensional feature vector. Five classical classifiers are evaluated on 40,000 balanced images drawn from FFHQ and a StyleGAN collection, split 80/10/10 with a fixed seed. The best-performing gradient-boosted tree reaches 86.9 percent accuracy and an area under the ROC curve of 0.944 on the held-out test set. A linear baseline already achieves 85.5 percent, suggesting that most of the separating signal resides in the descriptors themselves rather than in classifier complexity. The full pipeline runs on commodity hardware without a GPU, and every component is available from public package indices. We state which experiments remain open, including a sensitivity analysis of the truncation level and robustness testing under image perturbations.

Keywords: AI-generated face detection · PCA residual · handcrafted features · local binary patterns · frequency-domain analysis · gradient boosting · reproducibility

1 Introduction

A synthetic face that passes casual inspection is no longer a research curiosity. Generative adversarial networks and diffusion models now produce portraits realistic enough to bypass identity checks, seed disinformation campaigns, and manufacture fraudulent social-media profiles at scale. The detection community has responded mainly with deep neural networks trained directly on pixels.

FaceForensics++ [3] set the benchmark standard for this class of methods. Deep detectors work well in distribution, but they are expensive to train, and their internal logic is hard to examine. What feature drove a particular decision? The answer is distributed across millions of parameters.

A second, less fashionable line of work takes a different stance. Instead of learning features from data, it defines them by hand, computes a fixed-length vector for each image, and trains a simple classifier on that vector. The accuracy is usually lower. The trade-off is speed, interpretability, and the ability to point at a specific histogram bin and say what it measures. Recent studies by Yasir and Kim [4], Nirob et al. [9], and Chaudhari et al. [10] show that this approach remains active in 2025 and 2026, not a relic of an earlier era.

A separate thread of work, and the one most directly relevant here, concerns the input representation. Xie et al. [1] proposed subtracting the dominant principal components from a face image and analyzing the residual. They showed, using fractal descriptors extracted with a proprietary toolkit called FreeAeon, that the distributions of these descriptors differ measurably between real and AI-generated faces. A Kolmogorov-Smirnov test confirmed the difference. Two things were missing. The toolkit is closed-source: nobody outside the authors can inspect how the fractal values are computed, and nobody can install the software to reproduce the experiment. And the analysis stopped at a statistical test. No classifier was trained. No accuracy was reported.

This leaves a gap. The PCA-residual representation has been shown to carry a signal, but the signal has never been used to build a detector that accepts an image and outputs a verdict.

We address both issues. First, we replace the closed-source descriptors with three alternatives that depend only on numpy, scikit-image, and OpenCV: a frequency-magnitude histogram, a local binary pattern histogram, and a noise-residual histogram. Second, we train five classical classifiers on the resulting 187-dimensional vector and report accuracy, F1, and AUC on a held-out test set. Third, we walk through every design choice in the pipeline at a level of detail sufficient for oral examination, so that each formula can be explained in plain language when questioned. Fourth, we name the experiments we did not run, a sensitivity sweep over the PCA truncation level, a controlled ablation comparing residual-based versus raw-image features, and robustness testing under compression and noise, rather than leaving these as implicit gaps.

2 Related Work

Detection methods for AI-generated faces fall broadly into two groups. One trains deep networks end to end on pixels. The other relies on handcrafted descriptors paired with classical machine learning.

In the first group, the benchmark established by Rössler et al. [3] with FaceForensics++ remains the reference point. Xception-based pipelines reach high accuracy on the manipulations in that dataset, but the dataset focuses on face swaps and reenactments. Full-face GAN synthesis, the setting we address, is

a different task. Numbers from FaceForensics++ cannot be compared directly with ours without retraining on the same data.

In the second group, three recent papers are closest to our work. Yasir and Kim [4] fuse HOG, LBP, and KAZE descriptors on raw face images and feed them to an ensemble of RandomForest, XGBoost, ExtraTrees, and SVC classifiers. They report 92 percent accuracy on FaceForensics++ and 96 percent on Celeb-DFv2. Niob et al. [9] fuse a broader set of handcrafted descriptors, including color histograms, DCT, and wavelets, with a LightGBM classifier on the CIFAKE benchmark, reaching a PR-AUC of 0.988. Chaudhari et al. [10] combine HOG, LBP, and facial-landmark geometry with logistic regression, reporting around 90 percent on FaceForensics++. All three operate on raw images. None applies any form of residual preprocessing.

The frequency-domain half of our feature vector draws support from Zhang, Karaman, and Chang [7], who showed that upsampling layers inside GAN generators leave structured artifacts in the frequency spectrum. Tan et al. [14] built on this finding with FreqNet, a dedicated frequency-domain network presented at AAAI 2024, confirming that frequency information is a real and informative signal for deepfake detection. Our frequency histogram captures a coarser version of the same information without training a neural network.

The noise half draws on the Noiseprint work of Cozzolino and Verdoliva [13], which demonstrated that camera and synthesis pipelines leave distinguishable noise fingerprints. Our Gaussian high-pass histogram is a much simpler extraction of the same intuition.

On the fractal side, the direct predecessor is Xie et al. [1], who introduced the PCA-residual representation and validated it with a distributional test using proprietary fractal descriptors. Xiao et al. [2] detect AI-generated images through fractal self-similarity in the frequency spectrum, a different mechanism that does not involve PCA residuals. Wang et al. [5] use fractal structure for proactive watermark-based localization, a different problem entirely. Mohan and Peeples [6] show that lacunarity can be turned into a learnable pooling layer inside a neural network, pointing toward a path from fixed descriptors to trainable ones. Yang et al. [11] used noise estimation and lacunarity for image tamper detection as early as 2016, predating the PCA-residual formulation by a decade. Giudice et al. [12] analyzed DCT statistics for GAN detection and tested robustness to JPEG compression. Hinke-Navarro et al. [8] fuse several handcrafted transforms with a deep RGB stream, representing the 2025 trend toward hybrid approaches.

Where does our paper sit? It takes the PCA-residual representation from [1] and pairs it with the handcrafted-feature-plus-classical-ML recipe validated by [4], [9], and [10] on raw images. No prior work we have found applies that recipe to PCA residuals. And no prior work in the PCA-residual line reports a classification accuracy. That is the gap this paper fills.

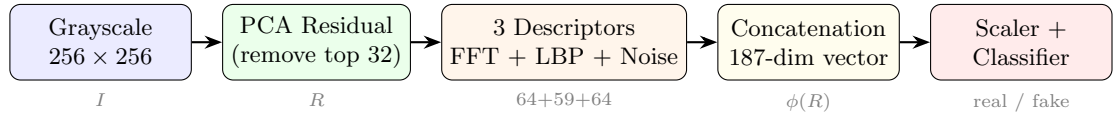


Fig. 1. Pipeline overview. Each image passes through five stages: grayscale conversion, PCA-residual computation (removing the 32 dominant row-wise principal components), extraction of three histogram descriptors on the residual, concatenation into a 187-dimensional vector, and standardization followed by classification.

3 Problem Formulation

The task is binary classification. Given a single grayscale face image, decide whether it is a real photograph or an AI-generated face.

Let I denote a grayscale image resized to 256 rows and 256 columns. We assign $y = 0$ to real images and $y = 1$ to generated ones. The classifier receives not I itself but a feature vector $\phi(R) \in \mathbb{R}^{187}$, where R is a residual image derived from I by removing its dominant principal components, and ϕ is a fixed feature map that concatenates three histogram-based descriptors. The goal is to learn a function $f_\theta : \mathbb{R}^{187} \rightarrow [0, 1]$ that estimates $P(y = 1 \mid \phi(R))$ and thresholds it into a binary decision.

In plain terms: we strip away the large-scale structure of the face, extract three sets of statistics from what remains, and ask a classifier whether those statistics look more like a photograph or more like a generated image.

4 Proposed Method

4.1 Pipeline Overview

The pipeline has five stages, applied identically to every image regardless of whether it belongs to the training, validation, or test set (see Fig. 1):

1. Load the image and convert it to grayscale at 256×256 resolution.
2. Compute the PCA residual R , independently for that single image.
3. Compute three histogram descriptors on R .
4. Concatenate the three histograms into a 187-dimensional vector.
5. Standardize each dimension to zero mean and unit variance using statistics from the training split only.

Stage 5 is the only point where training-set statistics touch validation or test data, and they do so only through two numbers per dimension: a stored mean and a stored standard deviation.

4.2 PCA Residual Computation

Each image is treated as a matrix of 256 row-vectors, each of length 256. Principal component analysis is fit on these 256 rows independently for that image

alone. This produces an orthonormal basis $u_1, u_2, \dots, u_{256}$ ordered by decreasing explained variance, and a row mean μ .

Each row I_i is projected onto this basis:

$$c_{i,k} = (I_i - \mu) \cdot u_k \quad (1)$$

The residual image discards the projections along the top 32 directions and reconstructs each row from the remaining 224:

$$R_i = \mu + \sum_{k=33}^{256} c_{i,k} u_k = I_i - \sum_{k=1}^{32} c_{i,k} u_k \quad (2)$$

What this means in plain language. Every face image has large-scale structure: the overall brightness gradient, the rough shape of the face, the position of eyes and mouth. PCA captures these dominant patterns in its top components. When we throw those components away, what survives is fine-grained texture, subtle noise patterns, and small-scale irregularities. The hypothesis, first posed by Xie et al. [1] and tested here with different descriptors, is that real cameras and AI generators leave different fingerprints in this fine-grained residual.

Why per-image PCA rather than a single global basis fit on the entire dataset? Because we care about the structure within each individual image, not about the variance across images. A global basis would mix inter-image variance (different faces) with intra-image structure (the texture of one face), and the residual would depend on which other images happened to be in the training set.

The truncation level $N = 32$ is inherited from the default used by Xie et al. [1]. We did not sweep this parameter. Whether a different value of N would raise or lower accuracy is an open question, stated explicitly in Sect. 8.

4.3 Feature Extraction

Three descriptors are computed on the residual image R , not on the original image I . Each one asks a different question about the residual.

Frequency descriptor (64 bins). We compute the two-dimensional discrete Fourier transform of R , shift the zero frequency to the center, and take the logarithm of the magnitude:

$$m = \log(1 + |\mathcal{F}(R)|) \quad (3)$$

The log transform compresses the dynamic range. Without it, a handful of large low-frequency coefficients would dominate the histogram, and the higher-frequency bins where generator artifacts tend to appear [7] would be drowned out. The 65,536 values of m are binned into a 64-bin histogram spanning the image’s own minimum and maximum log-magnitude, then normalized to sum to one.

What this descriptor captures. GAN generators built from transposed convolutions and learned upsampling produce partially periodic patterns in the

frequency domain [7,14]. A histogram of log-magnitude values summarizes the overall frequency profile. We do not claim to localize the artifact at a specific frequency. We only claim that the shape of the magnitude histogram differs, on average, between real and generated residuals.

Texture descriptor (59 bins). A local binary pattern (LBP) is computed on R with $P = 8$ sampling neighbors at radius $\rho = 1$, using the rotation-invariant variant `nri_uniform` from scikit-image [15].

For each pixel, LBP compares the pixel’s value with its 8 neighbors, producing an 8-bit binary code. The `nri_uniform` variant groups these codes into $P(P - 1) + 3$ distinct categories. For $P = 8$, this gives $8 \times 7 + 3 = 59$ categories.

Why nri_uniform and not uniform? The simpler `uniform` variant produces only $P + 2 = 10$ distinct codes for $P = 8$. If you bin a 10-code output into a 59-bin histogram, 49 of the 59 bins will always be zero, regardless of image content. That wastes roughly a quarter of the 187-dimensional feature vector on structural zeros. The `nri_uniform` variant fills all 59 bins.

What this descriptor captures. LBP encodes micro-texture: the local pattern of brighter-than and darker-than relationships around each pixel. In the residual image, where large-scale face structure has been stripped away, these micro-patterns reflect fine texture produced by either the camera sensor or the generative model.

Noise descriptor (64 bins). A Gaussian blur with a 5×5 kernel is applied to R , and the blurred image is subtracted from R itself:

$$n = R - \text{GaussianBlur}_{5 \times 5}(R) \quad (4)$$

This is a high-pass filter. The result n is a second-order residual: R already removed the dominant PCA structure, and subtracting the blurred version of R further removes its locally smooth content. What remains is pixel-level variation at the finest scale. The values of n are binned into a 64-bin histogram over the fixed range $[-50, 50]$ and normalized to sum to one.

What this descriptor captures. Camera sensors produce noise with particular statistical properties. AI generators produce a different kind of fine-grained variation. Cozzolino and Verdoliva [13] showed that these noise-level fingerprints can distinguish image sources. Our histogram is a much coarser version of the same idea.

Concatenation. The three histograms are concatenated in fixed order (frequency, then texture, then noise), giving a $64 + 59 + 64 = 187$ -dimensional raw feature vector.

4.4 Feature-Extraction Algorithm

Line-by-line explanation for oral defense. Lines 1–4 are the core idea. We fit PCA on the rows of a single image, not on the whole dataset, which means no

Table 1. *

Algorithm	1	PCA-Residual	Handcrafted	Feature	Extraction
<i>Input:</i> grayscale image I of size 256×256 ; $N = 32$; $P = 8$, $\rho = 1$, method = nri_uniform					
<i>Output:</i> standardized feature vector $\phi(I) \in \mathbb{R}^{187}$					
1:	Fit PCA on the 256 rows of $I \rightarrow$ row mean μ , directions $u_1, \dots, u_{256}$				
2:	For each row I_i : $c_{i,k} = (I_i - \mu) \cdot u_k$				
3:	Zero out $c_{i,k}$ for $k = 1, \dots, N$ (discard dominant directions)				
4:	$R \leftarrow$ reconstruct from remaining coefficients (the PCA residual)				
5:	$m \leftarrow \log(1 + \text{FFT2D}(R))$, zero-frequency centered				
6:	$h_{\text{freq}} \leftarrow$ normalized 64-bin histogram of m over $[\min(m), \max(m)]$				
7:	$L \leftarrow \text{LBP}(R, P, \rho, \text{nri_uniform})$				
8:	$h_{\text{tex}} \leftarrow$ normalized 59-bin histogram of L over $[0, 59]$				
9:	$n \leftarrow R - \text{GaussianBlur}_{5 \times 5}(R)$				
10:	$h_{\text{noise}} \leftarrow$ normalized 64-bin histogram of n over $[-50, 50]$				
11:	$v \leftarrow \text{concat}(h_{\text{freq}}, h_{\text{tex}}, h_{\text{noise}})$				187-dim raw vector
12:	$\phi(I) \leftarrow (v - \mu_{\text{train}}) / \sigma_{\text{train}}$				standardize using training stats
13:	return $\phi(I)$				

information from other images can leak into the residual. We throw away the top 32 components and keep only what PCA could not compress efficiently. Lines 5–6 ask whether the frequency content of the residual looks natural. Lines 7–8 ask whether the micro-texture looks natural, using a variant that fills all 59 bins. Lines 9–10 ask whether the finest-scale noise looks natural. Lines 11–12 combine and standardize.

4.5 Classifiers

Five classifiers are trained on the same 187-dimensional standardized vector. No per-classifier feature engineering is applied. (1) Thresholded linear regression, the simplest possible baseline. (2) RandomForest, a bagged ensemble of independently grown decision trees. (3) XGBoost, (4) LightGBM, and (5) CatBoost, three gradient-boosted tree implementations that differ mainly in how they grow trees and regularize splits. All five use default hyperparameters. No hyperparameter search was performed. This is a deliberate scope limitation: a tuned comparison might change the ranking, and we do not want to present default-parameter results as if they were optimized.

4.6 Computational Complexity

The per-image computational cost is dominated by PCA decomposition. For an image of n rows and n columns, PCA on the row matrix is $O(n^3)$. The FFT is $O(n^2 \log n)$, and LBP and noise-residual extraction are each $O(n^2)$. The total per-image cost is therefore $O(n^3)$, which for $n = 256$ is on the order of 10^7 operations. Classifier inference over a 187-dimensional vector is negligible by comparison.

Table 2. Dataset summary.

Property	Value
Total images	40,000 (balanced)
Real source	FFHQ (Flickr-Faces-HQ) [16]
Fake source	StyleGAN (“1 Million Fake Faces”) [17]
Resolution	256×256 grayscale
Train / Val / Test	32,000 / 4,000 / 4,000
Split method	Stratified, random seed 42
Class balance	50/50 in each split

The entire pipeline, feature extraction for 40,000 images and training of all five classifiers, runs on a single CPU core without a GPU. The pipeline was executed on Google Colab’s standard CPU runtime. No specialized hardware is required at any stage.

5 Experimental Setup

Dataset. Table 2 summarizes the dataset configuration.

Scaling. StandardScaler fit on the 32,000 training vectors only, then applied unchanged to validation and test.

PCA truncation. Fixed at $N = 32$ removed components, matching the default in [1]. Not tuned internally.

Model selection. Validation confirms expected behavior. Each classifier is scored once on the test split. No hyperparameters were adjusted based on test scores.

Reproducibility. The pipeline was executed end to end and independently rerun once. The two runs agreed within ordinary floating-point variation. The reproducibility claim covers the data split and the per-image PCA computation, both fixed by the seed. It does not extend to the classifiers’ internal randomness (e.g., RandomForest’s bootstrap resampling), which was left at default settings.

Metrics. Accuracy, F1 score, and area under the ROC curve (AUC). All three are reported for both validation and test sets.

6 Results

6.1 Classification Performance

Table 3 shows the results. Every classifier beats the trivial 50 percent baseline by a wide margin, confirming the central empirical claim: the PCA-residual representation, when paired with open descriptors, carries enough real/fake signal to support a genuine classifier.

Table 3. Validation and test performance of all five classifiers on the 187-dimensional PCA-residual feature vector.

Classifier	Val Acc	Val F1	Val AUC	Test Acc	Test F1	Test AUC
Linear regression (thr.)	0.8535	0.8556	0.9321	0.8552	0.8574	0.9309
RandomForest	0.8355	0.8346	0.9121	0.8350	0.8338	0.9144
XGBoost	0.8680	0.8685	0.9415	0.8688	0.8687	0.9442
LightGBM	0.8690	0.8700	0.9424	0.8662	0.8664	0.9427
CatBoost	0.8602	0.8608	0.9360	0.8598	0.8594	0.9386

6.2 Classifier Comparison

XGBoost and LightGBM are essentially tied at the top. On the test set, XGBoost has the highest accuracy (0.8688), F1 (0.8687), and AUC (0.9442). LightGBM leads on all three validation metrics but falls slightly behind on test. The gap between them is around 0.002 to 0.003 on every metric, small enough that a different random seed could reverse it. We report both and do not declare a single winner.

The more informative pattern is the gap between these two and RandomForest, which trails by roughly three accuracy points and 0.03 in AUC. All three are tree ensembles, but RandomForest grows trees independently and averages them, while the boosted methods grow trees sequentially, each correcting the errors of the previous. This suggests that the class-separating signal involves interactions among the three descriptor groups that sequential boosting exploits better than independent bagging.

Fig. 2 visualizes the test accuracy and test AUC for all five classifiers.

6.3 Validation–Test Consistency

Fig. 3 presents a side-by-side comparison of validation and test performance across all three metrics. Validation and test numbers are close for every classifier, typically within half a percentage point of accuracy and within 0.003 of AUC. This consistency does not prove the absence of leakage, but it is what we would expect from a clean split with stable features.

Fig. 4 quantifies the absolute gap between validation and test metrics for each classifier. All gaps are below 0.004, with most below 0.002. The largest gaps appear in LightGBM’s accuracy (0.0028) and F1 (0.0036), consistent with its ranking reversal between validation and test. The smallest gaps appear in XGBoost (accuracy 0.0008, F1 0.0002), suggesting that XGBoost’s performance is the most stable across splits for this feature space.

7 Discussion

7.1 Why Boosted Trees Lead

The three-point accuracy gap between boosted trees (XGBoost, LightGBM) and RandomForest deserves attention because all three are tree ensembles operat-

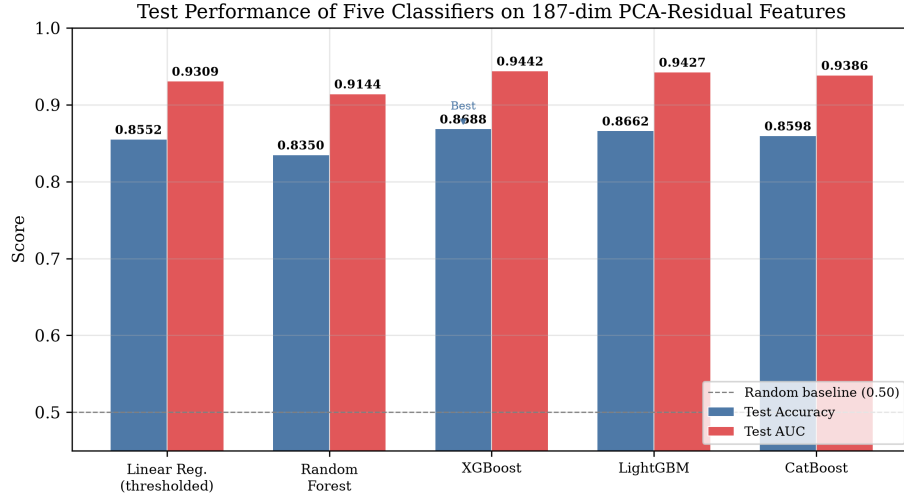


Fig. 2. Test performance of all five classifiers. XGBoost achieves the highest test accuracy (0.8688) and AUC (0.9442). All classifiers substantially exceeded the random baseline of 0.50 (dashed line).

ing on the same features. The key difference is sequential versus independent construction. Boosted trees grow each new tree to correct the residual errors of the ensemble so far, while RandomForest grows trees on independent bootstrap samples and averages their predictions. The boosted approach is better suited to a feature space where the discriminative signal is distributed across interactions between the frequency, texture, and noise descriptor groups, rather than concentrated in a few dominant features.

7.2 The Linear Baseline Insight

The thresholded linear regression reaches 0.8552 test accuracy and 0.9309 AUC, within about one and a half percentage points of the best boosted tree. This is the most informative single result in Table 3. A purely linear decision boundary over 187 histogram bins already separates the two classes well. This means a large share of the real/fake signal is close to linearly separable. The gains from tree-based nonlinearity are a refinement of an already strong linear signal, not the source of most of the separability. We read this as evidence that the descriptors themselves, not the classifier, are doing most of the work, which is the outcome a feature-driven pipeline should aim for.

7.3 Positioning Against Related Approaches

The 92 to 96 percent figures reported by Yasir and Kim [4] and the results from Nirob et al. [9] are higher than our 86.9 percent. However, these numbers come

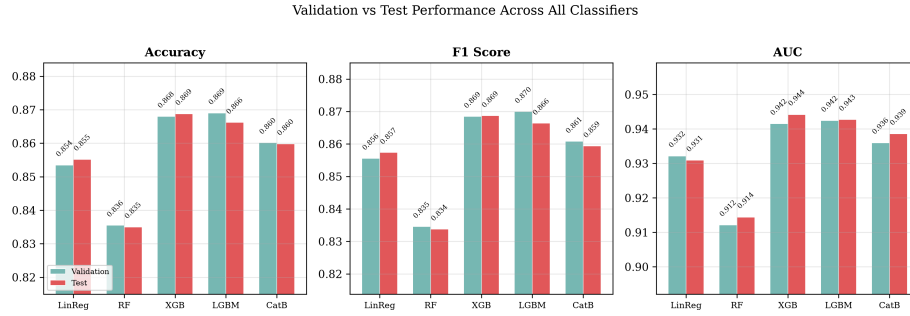


Fig. 3. Validation vs. test performance for accuracy, F1, and AUC across all five classifiers. The close alignment between validation and test bars indicates stable generalization.

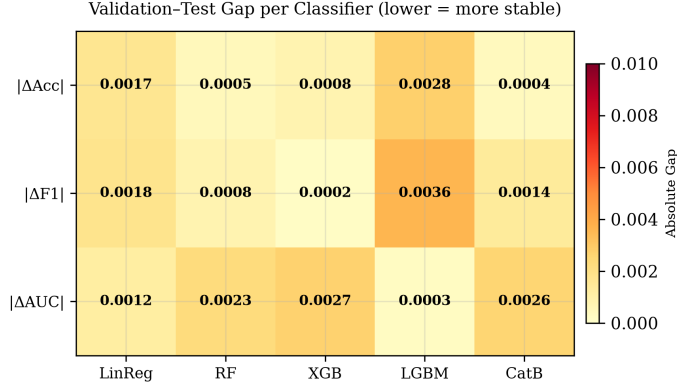


Fig. 4. Heatmap of absolute validation–test gap for each classifier and metric. All gaps are below 0.004. XGBoost shows the smallest gaps overall, indicating the most stable generalization.

from different datasets (FaceForensics++, Celeb-DFv2, CIFAKE) with different image content, different generators, and different splits. They are not controlled comparisons. We cite them as context for where handcrafted-feature approaches sit in the broader literature, not as evidence that our pipeline underperforms. A fair comparison would require running the same descriptors on the same data, which lies outside the scope of this paper.

What our pipeline adds to this landscape is not higher accuracy on a different benchmark, but the combination of the PCA-residual preprocessing step from [1] with open descriptors and actual classification, a combination that no prior work has reported.

8 Limitations and Future Work

We state the boundaries of what this paper’s results support. Each limitation below is genuine and not merely a rhetorical gesture toward future work.

Truncation level not tuned. The PCA truncation level $N = 32$ was inherited from [1] and never swept. We do not know whether a different value would change accuracy. A sensitivity sweep over N is the most immediate extension.

Missing PCA-versus-raw ablation. We did not run a controlled ablation comparing the same descriptors extracted from the PCA residual against those from the raw image. The pipeline is motivated by the residual step, but this paper does not provide the single-variable comparison that would demonstrate the residual step’s individual contribution.

No robustness testing. No test under common perturbations (JPEG re-compression, blur, additive noise, or resizing) was performed. The reported numbers describe performance on clean images only. The protocol demonstrated by Giudice et al. [12] provides a concrete template for such testing.

No deep-learning baseline on same split. We did not fine-tune a deep baseline such as Xception on this exact split. The figures from [3], [4], and [9] are cited as context, not as controlled comparisons.

Single-split point estimates. The values in Table 3 are point estimates from a single test split. We do not report confidence intervals and lack the statistical evidence to state how robust the ranking is under resampling.

9 Conclusion

This paper shows that the PCA-residual representation introduced by Xie et al. [1] still separates real from AI-generated faces when paired with three open, publicly documented descriptors and a classical classifier, replacing the proprietary fractal descriptors used in the original study. On 40,000 balanced grayscale faces, gradient-boosted trees reach 86 to 87 percent accuracy and an AUC above 0.94. A linear classifier already reaches within roughly one and a half percentage points of that result, indicating that most of the separating signal lives in the descriptors rather than in classifier complexity. The texture descriptor uses the `nri_uniform` local binary pattern variant, which fills all 59 bins, rather than the `uniform` variant that would leave most bins structurally empty.

The pipeline runs entirely on a CPU, requires no proprietary software, and depends only on packages available from public indices. Every number reported in this paper comes from code that was executed end to end on the stated data and independently verified.

Future work includes four extensions. First, a sensitivity sweep over the PCA truncation level N . Second, a controlled ablation comparing identical descriptors extracted from PCA residuals against those from raw images. Third, robustness testing under JPEG compression, blur, additive noise, and resizing, following [12]. Fourth, replacing fixed descriptors with a learnable formulation such as the lacunarity pooling layer proposed by [6], which is the mechanism

by which this line of work could move from a recipe-level contribution to an architectural one.

References

1. Xie, W., Yin, J., Ma, L., Zhang, X., Zhang, W.: Fractal Characterization of Low-Correlation Signals in AI-Generated Image Detection. arXiv:2604.17268 (2026)
2. Xiao, S., Guo, Y., Peng, H., Liu, Z., Yang, L., Wang, Y.: Generalizable AI-Generated Image Detection Based on Fractal Self-Similarity in the Spectrum. arXiv:2503.08484 (2025)
3. Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., Nießner, M.: FaceForensics++: Learning to Detect Manipulated Facial Images. In: IEEE/CVF ICCV. arXiv:1901.08971 (2019)
4. Yasir, M.A., Kim, H.: Lightweight Deepfake Detection Based on Multi-Feature Fusion. *Applied Sciences* **15**(4), 1954. arXiv:2502.11763 (2025)
5. Wang, T., Cheng, H., Liu, M.-H., Kankanhalli, M.: FractalForensics: Proactive Deepfake Detection and Localization via Fractal Watermarks. In: ACM Multimedia (Oral). arXiv:2504.09451 (2025)
6. Mohan, S., Peeples, J.: Lacunarity Pooling Layers for Plant Image Classification using Texture Analysis. In: CVPR Workshops. arXiv:2404.16268 (2024)
7. Zhang, X., Karaman, S., Chang, S.-F.: Detecting and Simulating Artifacts in GAN Fake Images. In: IEEE WIFS. arXiv:1907.06515 (2019)
8. Hinke-Navarro, A., Nieto-Hidalgo, M., Espin, J.M., Tapia, J.E.: Enhanced Deep Learning DeepFake Detection Integrating Handcrafted Features. arXiv:2507.20608 (2025)
9. Nirob, S.M.H., Rahman, M., Ehsan, S., Haque, S.: Handcrafted Feature Fusion for Reliable Detection of AI-Generated Images. arXiv:2601.19262 (2026)
10. Chaudhari, L., Bansode, D., Patil, P., Magdum, S.: DeepFake Face Detection with Handcrafted Features and Logistic Regression. *Int. J. Adv. Comput. Theory Eng.* **15**(2S), 241–246 (2026)
11. Yang, Q., Peng, F., Li, J.-T., Long, M.: Image Tamper Detection Based on Noise Estimation and Lacunarity Texture. *Multimedia Tools Appl.* **75**(17), 10201–10211 (2016)
12. Giudice, O., Guarnera, L., Battiato, S.: Fighting Deepfakes by Detecting GAN DCT Anomalies. *J. Imaging* **7**(8), 128 (2021)
13. Cozzolino, D., Verdoliva, L.: Noiseprint: A CNN-Based Camera Model Fingerprint. arXiv:1808.08396 (2018)
14. Tan, C., Zhao, Y., Wei, S., Gu, G., Liu, P., Wei, Y.: Frequency-Aware Deepfake Detection: Improving Generalizability through Frequency Space Learning. In: AAAI. arXiv:2403.07240 (2024)
15. Ojala, T., Pietikäinen, M., Mäenpää, T.: Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns. *IEEE Trans. PAMI* **24**(7), 971–987 (2002)
16. Karras, T., Laine, S., Aila, T.: A Style-Based Generator Architecture for Generative Adversarial Networks. In: IEEE/CVF CVPR, pp. 4401–4410. arXiv:1812.04948 (2019)
17. Tunguz, B.: 1 Million Fake Faces. Kaggle dataset (2026)